# Embodied Multimodal World Modeling with 4D Scene Graphs for Long-Distance Teleoperation

Zeyu Xue[1], Zekai Lin[1], Siyuan Ye[1],
Yi Wang[1], Mujie Liu[1], Shan An[2], Zhenyu Lu[1*], Chenguang Yang[3]

***Abstract*—Long-distance robot operation inevitably suffers from network delay and temporary disconnections, leading to a persistent mismatch between human-issued instructions and the robot's actual environment during task execution. In such settings, simply pausing execution or relying on static perception is impractical. This paper presents 4D-OVSG, an embodied multimodal world modeling framework that enables robust task execution under delay. The core idea is to compress delayed RGB-D video streams into a compact 4D open-vocabulary scene graph. The framework introduces a GMM–based keyframe extraction module to reduce computational redundancy, together with a Multi-Object Tracking (MOT) pipeline to maintain stable object identities over time. Furthermore, a hierarchical planning strategy based on VLMs/LLMs is employed to validate instructions and detect environment change. Real-robot experiments show that 4D-OVSG achieves reliable task execution with up to 68.9% success rate while maintaining inference times as low as 79.89 s. Compared to ST-OVSG, our approach achieves over 7× faster inference with higher success rates in complex dynamic scenarios. This work provides a robust semantic foundation for long-distance task-level teleoperation.**

***Index Terms*—Embodied multimodal intelligence, scene graph, world modeling, planning, communication latency, robotics**

## I. Introduction

Teleoperated robots play a crucial role in extreme and remote environments such as space station maintenance, lunar surface operations, and deep-sea exploration. In these scenarios, a human operator controls a distant manipulator via RGB-D video streamed to a local console, effectively turning the system into an interactive multimedia application with an embodied agent at the far end. However, in realistic wide-area networks, end-to-end communication delay and temporary disconnections are unavoidable [1]. This problem is particularly severe in ultra-long-distance settings. Due to fundamental limits in communication speed and engineering structure, the round-trip latency can reach several minutes. This latency creates a persistent mismatch between the state observed by the operator and the actual state around the robot—in other words, commands always lag behind the environment. In such regimes, action-level teleoperation becomes impractical and systems must shift to task-level teleoperation, where the robot is expected to have a certain degree of autonomy. However, even at the task level, the inconsistency between delayed instructions and the current environment remains a major bottleneck for overall performance.

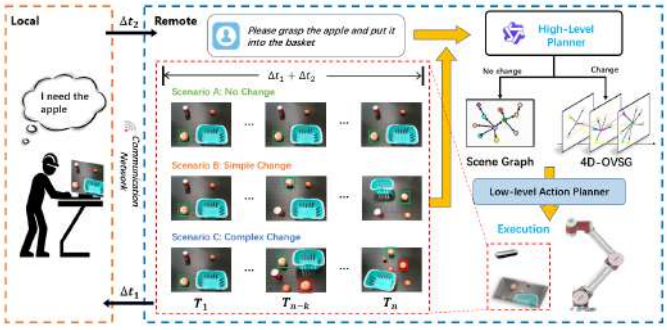



Fig. 1. **Problem Formulation**. The **Remote** robot captures a camera frame $I(T_1)$ at time $T_1$ , which is sent to the **Local** operator with downlink latency $\Delta t_1$. Then, the operator issues a natural-language command $u$ based on $I(T_1)$, and this command is transmitted back to the robot with uplink latency $\Delta t_2$. During the interval $[T_1, T_n]$, environment state $s(t) \in \mathcal{S}$ may evolve under three scenarios: Scenario A (static), Scenario B (simple change), and Scenario C (complex change). These changes may lead to a mismatch between instructions and environment.

Most existing work mitigates these issues mainly from two directions. Classical methods compensate delay by improving communication and hardware: higher-bandwidth links, better video codecs, or more onboard compute to accelerate perception and control [2], [3], [4]. From a multimedia perspective, these approaches primarily focus on optimizing transmission and low-level processing of video streams. But they do not address a fundamental question: *How should the system interpret and leverage delayed information for planning?*

Recently, Foundation Models (LLMs/VLMs) have shown strong reasoning and generalization capabilities [5]. Many works have begun to integrate these models into systems to enhance robotic autonomy and high-level task planning in open environments [6], [7], [8], [9]. These approaches have achieved promising results on multi-step manipulation and navigation in semi-structured or even unknown scenes. However, most LLM-based planners assume real-time environmental consistency and fail to account for the temporal "state-gap" caused by delay. Therefore, when such models are directly deployed in teleoperation settings, they may issue instructions based on outdated visual feedback, leading to execution failures in dynamic scenes.

To address these issues in a more systematic way, we propose ***4D-OVSG*** (4D Open-Vocabulary Scene Graph), a novel

This work was supported by the Fundamental Research Funds for the Central Universities 2025ZYGXZR057 and the Guangdong Provincial Key Area R&D Program 202580909030001.

[1]HI-Robot Lab, School of Automation Science and Engineering, South China University of Technology.
[2]School of Electrical and Information Engineering, Tianjin University
[3]School of Computer Science and Informatics, University of Liverpool.
*Corresponding author

framework for task-level teleoperation under communication latency. The key idea is to couple high-level hierarchical planning with a structured 4D World Model. At the algorithmic level, we design an image–video processing pipeline that compresses the history RGB-D stream into a single 4D Scene Graph. On the planning side, VLMs/LLMs are used to assess the validity of the instruction and generate action sequences on top of this structured world model. The resulting 4D Scene Graph encodes the spatio-temporal evolution of the scene and serves as a structured world model for task-level planning under communication latency.

The main contributions of this work can be summarized as follows:

1) **Latency-Robust Teleoperation via 4D World Modeling:** We propose 4D-OVSG, a novel framework designed to bridge the spatio-temporal gap between local instructions and remote scene states under communication delays. By coupling a structured spatio-temporal world model with hierarchical planning, the system can validate and adapt delayed instructions to the current environment, ensuring task execution robustness in dynamic settings.
2) **Efficient 4D Scene Construction:** We introduce an efficient 4D scene graph construction pipeline that reduces temporal redundancy and improves cross-frame consistency. By introducing keyframe selection to eliminate temporal redundancy, MOT-based temporal edges to maintain object identity over time, and a geometry-aware polar spatial encoding, our method achieves both higher robustness in dynamic scenes and substantially lower inference cost.
3) **Real-robot validation:** We conduct extensive real-robot experiments across multiple task settings and latency regimes, demonstrating that 4D-OVSG consistently improves task success rates while maintaining lower inference time.

## II. Related Work

### A. FMs for Robot Perception and Planning

Recent work leverages Foundation Models (FMs) for flexible perception. VLM-based detectors and segmentation models (e.g. CLIP-based models [10], Grounding DINO [11], SAM/SAM2 [12], RAM [13]) enable open-vocabulary object recognition and detection in images. These systems excel at identifying "what" objects are and "where" they are in a single frame [14], but they do not enforce temporal consistency or consider communication delays. In other words, current VLM methods assume a static scene snapshot and do not reason about how objects moved between frames.

### B. Semantic Scene Representations

Traditional spatial representations—such as occupancy grids, TSDF volumes, static semantic maps—primarily focus on geometry or per-pixel labels and largely ignore object-level temporal relations. Classic SLAM methods build geometric maps annotated with class labels [15]–[17], but once objects move the map becomes outdated. These static maps cannot answer queries about when or how objects moved.

To capture object-centric structure, scene graphs have been explored: 3D scene graphs and visual scene graphs represent objects as nodes and spatial relations as edges [18]. Recent open-vocabulary methods let the robot recognize novel objects and store language labels. For instance, NLMap [19] and VLMaps [20] fuse object-level visual features, text labels, and 3D coordinates, enabling language-driven queries. ConceptGraphs [21] integrate open-set object detections into a 3D scene graph for perception and planning. Graph-based representations have also been explored to support incremental and temporally consistent perception in long-running robotic systems [22]. These works improve generalization but are still static: they encode a scene snapshot and do not update as objects move or as time passes.

### C. 4D Scene Graphs for Robotics

4D Scene Graphs aim to address this gap by linking object nodes across time [18] . In teleoperation contexts, a recent approach –Spatio-Temporal Open-Vocabulary Scene Graph (ST-OVSG) [23]–explicitly attaches each object node with a timestamp (a "latency tag") and links instances across frames. It has shown improved task accuracy and robustness under latency and dynamic environments. However, ST-OVSG remains limited by a slow, heavyweight inference pipeline that processes the full history video, leading to temporal redundancy and high compute/memory cost. In addition, its cross-frame association relies on per-frame Hungarian matching, which is unstable under occlusion or similar objects, and its spatial relations are inferred solely by a large VLM, making them slow and prone to hallucinations. To address these issues, we propose 4D-OVSG, which introduces a keyframe extraction module to reduce computational redundancy and replaces frame matching with a MOT-based method, which is more stable and reliable temporal correspondence.

## III. Method

The problem formulation is shown in Fig. 1. To tackle these challenges, we propose a novel framework, which mitigates the effects of latency from two perspectives. First, a hierarchical planning strategy is employed to enhance the robot's autonomous decision-making and planning capabilities. Second, a 4D Scene Graph is established to help the system perceive spatial and temporal information. The overall architecture is shown in the Fig. 2.

### A. High-Level Planning

Due to the unavoidable latency, the validity of a command cannot be guaranteed when the environment has changed significantly during the interval $[T_1, T_\mathrm{n}]$. At the same time, the communication loop is computationally expensive; blindly reconstructing a full 4D scene graph even under minor environmental perturbations would lead to redundant consumption of computational resources.

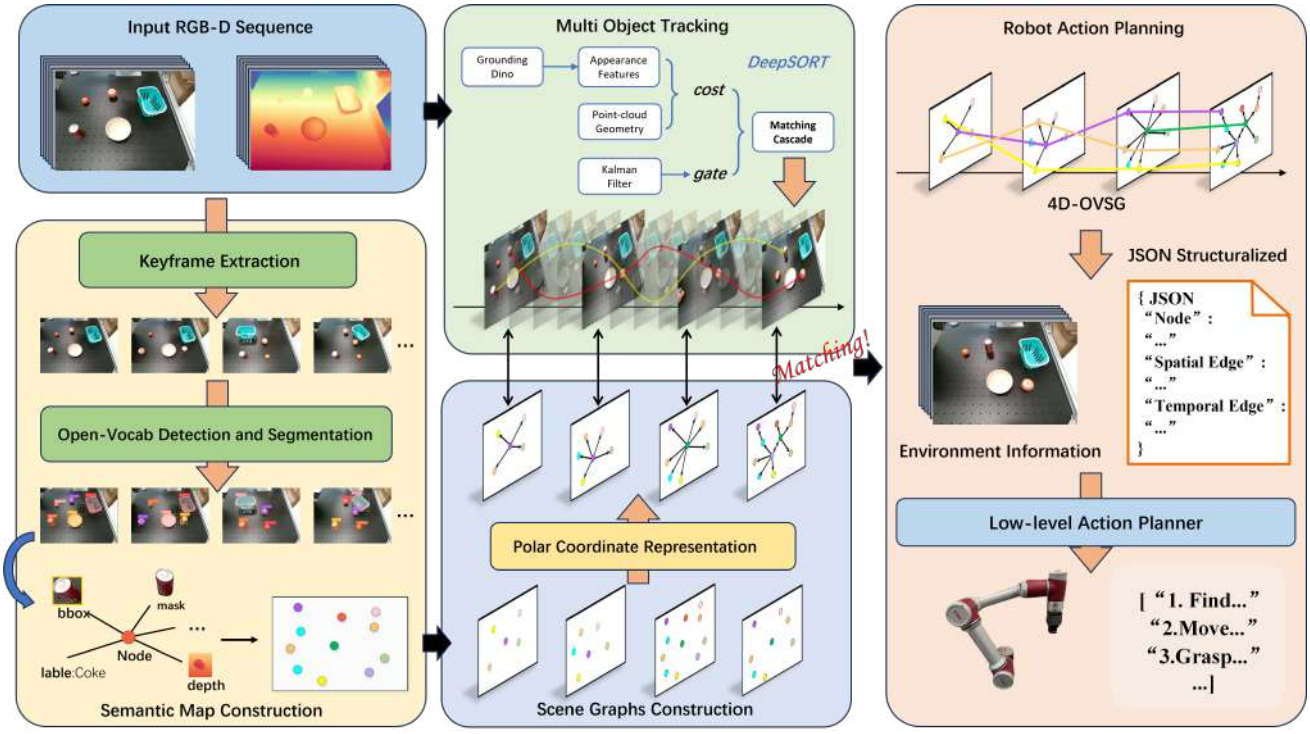



Fig. 2. Overview of our ***4D-OVSG*** System. The process begins with an input RGB-D sequence, which undergoes keyframe extraction to eliminate temporal redundancy. In the perception stage, open-vocabulary models perform object detection and segmentation to define semantic nodes. These nodes are organized into single-frame scene graphs using a polar-coordinate-based spatial representation for geometric accuracy. Simultaneously, a multi-modal MOT module establishes stable temporal edges across frames. Finally, the resulting 4D scene graph is structuralized into a compact JSON format, serving as a symbolic world model for the low-level LLM to generate precise, executable robot action sequences.

Therefore, we introduce a VLM with decision-making and reasoning capabilities as the core of the high-level planning module. On the one hand, it evaluates whether the current command is still feasible by jointly considering the current environment and the original instruction. Specifically, current environmental observations and the natural language instruction are fed into a Vision-Language Model (VLM). Through prompt engineering, the VLM is tasked with parsing the instruction and comparing it against the detected environmental state to determine command validity, while simultaneously generating a set of candidate category class labels $\{c_{i,t}\}, i \in \{1, \ldots, N\}$. If the VLM deems the command invalid, the system halts and requests a new instruction from the operator.

On the other hand, before entering the graph construction, we perform a fast environment change check to avoid building a full 4D scene graph when the scene has barely changed. Two frames were compared: frame $I(T_1)$ and current frame $I(T_{\mathrm{n}})$. After converting both frames to grayscale and applying Gaussian smoothing, we compute the absolute pixel difference and apply a fixed threshold to obtain a binary change mask $M(x, y)$. Simple morphological operations are then applied to reduce isolated noise. Based on this mask, the change ratio is defined as:

$$\rho_{\mathrm{chg}} = \frac{1}{HW} \sum_{x=1}^{W} \sum_{y=1}^{H} M(x, y), \tag{1}$$

where $H$ and $W$ are the height and width of the image, and $\rho_{\mathrm{chg}} \in [0, 1]$ measures the fraction of pixels classified as changed. Then, a simple binary decision rule is applied to determine whether the environmental changes are significant enough to warrant the construction of a full 4D-SG.

### *B. Multi-Object Tracking*

Instead of relying on fragile cross-frame geometric matching, we adopt a modern multi-object tracking (MOT) pipeline built on DeepSORT [24], which is designed for long video sequences and explicitly models track birth, death, and occlusion. At the detection level, GroundingDINO [11] is used as an open-vocabulary detector to handle novel scenes and long-tail categories. At the tracking level, we retain DeepSORT's design, but extend the association cost to fuse three modalities—appearance, motion, and point-cloud geometry:

$$c_{i,j} = \alpha\, \mathrm{cost}_{\mathrm{appearance}} + \beta\, \mathrm{cost}_{\mathrm{motion}} + \gamma\, \mathrm{cost}_{\mathrm{pointcloud}} \tag{2}$$

where $\alpha, \beta, \gamma > 0$ and $\alpha + \beta + \gamma = 1$. All three components are normalized to $[0, 2]$ so that their scales are comparable. For the **appearance cost**, a DeepSORT re-ID network extracts 2D appearance features, and each track maintains a gallery of recent features. Let $\mathbf{A}_i, \mathbf{B}_j \in \mathbb{R}^M$ be the feature vectors of track $i$ and detection $j$. After $\ell_2$-normalization, $\mathrm{cost}_{\mathrm{appearance}} = 1 - \mathbf{A}_i'^{\top} \mathbf{B}'_j$. For the **motion cost**, we use the Mahalanobis distance $d^2_{\mathrm{maha}}(x, \mu, \Sigma) = (x - \mu)^{\top} \Sigma^{-1} (x - \mu)$ between the Kalman-predicted state and the detection. Through compression, $\mathrm{cost}_{\mathrm{motion}} = \min(d^2_{\mathrm{maha}}/10,\, 2.0)$. For the **point-cloud**

**cost**, given 3D centroids $c_t, c_d$, point densities $d_t, d_d$, and dispersions $\sigma_t, \sigma_d$ of the track and detection, we define:

$$\text{cost}_{\text{pointcloud}} = \lambda_c \frac{\|c_t - c_d\|_2}{10} + \lambda_d |d_t - d_d| + \lambda_\sigma \frac{|\sigma_t - \sigma_d|}{\max(\sigma_t, \sigma_d, 1.0)}, \quad (3)$$

**Cascaded matching.** The system follows DeepSORT's cascaded matching strategy. Tracks with smaller age (recently confirmed) are matched first, solving a series of linear assignment problems over age-based subsets and iteratively updating the matched pairs and remaining detections. Finally, IoU-based matching is applied to age-1 tentative tracks to handle sudden appearance changes, point-cloud noise, and initialization errors. This design improves track stability under long-term occlusions and complex dynamic environments.

The MOT module outputs stable track IDs for each object across the video, which are directly used to construct temporal edges in the 4D scene graph.

### C. 4D Scene Graph

We propose a novel 4D scene graph construction scheme that compresses the delayed history video into a compact graph representation. This representation greatly reduces data volume while preserving most of the task-relevant information in the history, enabling the robot to exploit its "memory" to mitigate the impact of latency and improve task success rate.

*1) Historical memory:* The system uses a RealSense D455 camera with a fixed pose to capture an RGB-D sequence $\mathcal{D} = \{(I_t^{\text{rgb}}, I_t^{\text{depth}})\}_{t=1}^{n}$. The video recorded during the latency period is stored in a memory buffer and serves as the raw data for subsequent keyframe extraction and 4D scene graph construction.

*2) Keyframe Extraction:* Video streams typically contain substantial temporal redundancy, while the robot's onboard storage and compute budgets are highly constrained. Feeding the entire history video into downstream modules leads to long latency and excessive computational cost for 4D scene graph construction and planning. Therefore, the system introduces a keyframe extraction module based on a Gaussian Mixture Model (GMM), which uses background modeling and motion saliency to select a compact set of representative frames.

Keyframes are selected via a two-pass motion-based procedure. First, background subtraction is applied to estimate a per-frame motion magnitude $A_t$, and an adaptive threshold $\tau_{\text{area}}$ is derived to distinguish static and motion frames. In the second pass, motion segments are detected when $A_t \geq \tau_{\text{area}}$ persists longer than $L_{\min}$. A small number of frames are then uniformly sampled from each segment, while always including the first and last frames, yielding the keyframe set

$$\mathcal{K} = I_{t_1}, I_{t_{k'}}, \ldots, I_{t_{K'}}, \qquad T_1 = t_1 < \cdots < t_{K'} = T_n, \quad (4)$$

where $|\mathcal{K}| \ll T_n$ while preserving most motion-relevant information.

*3) Single-Frame Scene Graph Construction:*

*a) Node:* After keyframe extraction, a scene graph is constructed independently for each selected keyframe $I_{t_{k'}}$. For better generalization and recognition in novel scenes, the detection module uses a composite of open-vocabulary models. Concretely, for each keyframe, the Recognize-Anything Model (RAM) [13] is applied to predict a set of candidate category labels $\{c_{i,t}\}, i \in \{1, \ldots, M\}$, which serve as a supplement to the primary classes identified by the high-level planning module. These categories are then used as textual prompts for the open-vocabulary detector GroundingDINO [11] to produce category-aware bounding boxes $\{b_{i,t}\}_{i=1}^{M}$, which are further refined into object masks $\{m_{i,t}\}_{i=1}^{M}$. Each detected object is encoded using CLIP [10] to obtain a visual feature $\mathbf{f}_{i,t}^{\text{image}} \in \mathbb{R}^d$. Meanwhile, using the depth image and camera calibration, a corresponding object-level point cloud $\mathcal{P}_{i,t} \subset \mathbb{R}^3$ is extracted for each instance. Formally, a scene-graph node is defined as

$$n_{k,t} = \big(c_{k,t},\ b_{k,t},\ m_{k,t},\ \mathbf{f}_{k,t}^{\text{image}},\ \mathcal{P}_{k,t}\big), \quad (5)$$

which serves as the fundamental entity for subsequent spatial and temporal reasoning.

*b) Spatial edges:* Spatial relations are constructed in a key-object-centric manner. We first use CLIP to match the user instruction with candidate object categories in the current frame, and select the category with the highest similarity as the *core object class*. When multiple instances exist, the one closest to the manipulator workspace is selected as the final key object and used as a polar reference center for spatial edges to other nodes.

Let $\mathbf{p}_i = (x_i, y_i, z_i)^\top$ and $\mathbf{p}_j = (x_j, y_j, z_j)^\top$ denote the 3D positions of objects $i$ and $j$ in the robot base frame. Their relative spatial relation is encoded in polar form:

$$r_{ij} = \big\|\mathbf{p}_j - \mathbf{p}_i\big\|_2, \qquad \phi_{ij} = \text{atan2}(y_j - y_i,\ x_j - x_i), \quad (6)$$

The pair $(r_{ij}, \phi_{ij})$ is attached as the attribute of the directed spatial edge from node $n_{i,t}$ to $n_{j,t}$.

This geometry-aware representation provides an explicit and lightweight encoding of object relations, avoiding ambiguous language-based descriptors and costly learning-based relation predictors. As a result, the spatial edges are robust to viewpoint and scale variations, naturally aligned with the robot coordinate frame, and directly applicable to downstream motion planning and control.

*4) Temporal Edge Construction:* To capture how objects persist and evolve over time, temporal edges are introduced into the scene graph to connect the same physical object across adjacent keyframes. Unlike ST-OVSG, which relies on heuristic cross-frame matching, 4D-OVSG uses stable track IDs produced by the MOT module. Each keyframe is aligned with the MOT tracks at the same timestamp. For every detected object node in the keyframe $I_t$, the system computes the IoU between its bounding box $b_{k,t}$ and the bounding boxes $\hat{b}_{\tau,t}$ of all active tracks $\tau$ in that frame. If the IoU exceeds a threshold, the node is considered to correspond to track $\tau$ and the track ID is written into the node attributes. In addition, the visual

features $\mathbf{f}_{i,t}^{\text{image}} \in \mathbb{R}^d$ extracted at the node level are retained to support consistent state propagation along each track.

In this way, all nodes on the keyframes are attached to their underlying trajectories. Finally, nodes with the same track ID are linked to form temporal trajectories. Along each trajectory, object states—including geometric attributes and appearance features—are propagated forward, enabling the system to reason about object persistence, disappearance, and reappearance in delayed scenarios.

## IV. Experiments

### A. Implementation Details and Task Setup

*1) Implementation Details:* All experiments are conducted on a real robotic system. The setup uses a JAKA S 5 manipulator as the execution platform, equipped with a fixed-pose Intel RealSense D455 RGB-D camera for environment perception. The high-level planning module uses qwen3-vl-plus for instruction validation and intent inference, while the low-level action generation module adopts qwen3-omni-flash to translate scene-graph representations into executable action plans [25]. For fair comparison, all baselines share the same hardware, camera setup, and language models; only the perception and planning pipelines differ across methods.

*2) Tasks and Delay Settings:* To evaluate robustness under communication latency and dynamic environments, we design a set of tabletop manipulation tasks spanning three scene types: kitchen, office, and mixed clutter. Each scene includes multiple difficulty levels and natural-language instructions covering single-step and multi-step object manipulation.

We consider three latency settings to simulate realistic long-distance teleoperation: **(i) No Change**, where the environment remains static during the delay; **(ii) Simple Change**, involving minor object displacements or additions while the target remains accessible; and **(iii) Complex Change**, featuring significant scene alterations such as object disappearance or visually similar distractors.

Under this setup, each method is evaluated on all instructions across all scene types and delay regimes, resulting in a total of 135 real-robot task instances (3 scenes $\times$ 3 latency regimes $\times$ 15 instructions).

*3) Real robot experiment:* As shown in Fig. 3, task instructions are issued based on an outdated observation $I(T_1)$, while execution starts at a later time $T_n$. During the delay window, the robot continuously receives visual observations, while controlled perturbations are introduced to the scene to simulate realistic environment changes. The collected observations are later used to construct a 4D scene graph for reasoning at the execution time. Results demonstrate that the robot successfully placed the correct apple into the basket across all three delay conditions.

### B. Comparison with SOTA methods

To evaluate 4D-OVSG under long communication delay, we compare it with ConceptGraph and ST-OVSG across the three latency regimes described above. All methods are evaluated under identical hardware, task settings, and experimental conditions. Performance is measured using two metrics: **Success Rate (SR)**, defined as the proportion of instructions successfully executed by the robot, and **Inference Time (IT)**, defined as the time required to generate the final robot action sequence from a given instruction.

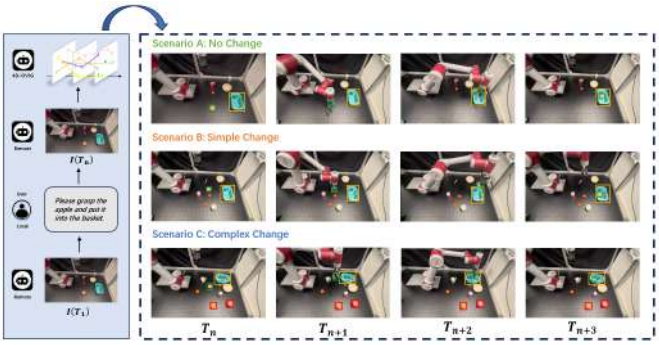



Fig. 3. Qualitative real-robot results under long communication delay. Task instructions are issued based on an outdated observation $I(T_1)$ and executed at a later time $T_n$. The proposed 4D-OVSG enables temporally consistent reasoning, allowing correct execution under no change, adaptive execution under simple changes, and robust handling of complex scene evolution.

TABLE I
Overall Performance Comparison across different scene change configurations.

| Method | Latency Config | Avg. Time (s) ↓ | Success Rate ↑ |
|---|---|---|---|
| ConceptGraph [21] | No Change | **17.98** | **95.6%** |
| ST-OVSG [23] | | 540.02 | 93.3% |
| **4D-OVSG (Ours)** | | 29.76 | 93.3% |
| ConceptGraph | Simple Change | **19.11** | 17.8% |
| ST-OVSG | | 611.39 | 63.7% |
| **4D-OVSG (Ours)** | | 96.28 | **66.7%** |
| ConceptGraph | Complex Change | - | - |
| ST-OVSG | | 839.33 | 33.3% |
| **4D-OVSG (Ours)** | | **113.65** | **46.7%** |

As demonstrated in Table I, while all methods perform comparably in static environments, 4D-OVSG and ConceptGraph (CG) exhibit significantly higher computational efficiency than ST-OVSG. However, as environmental dynamics increase in complexity, the success rate of CG deteriorates sharply and becomes ineffective in the most complex scenarios, due to its reliance on static spatial priors. Conversely, although ST-OVSG maintains a degree of robustness, its prohibitive inference time renders it impractical for real-world applications. In contrast, 4D-OVSG achieves a superior balance between robustness and efficiency, outperforming ST-OVSG in success rate by 13.4% in complex settings while reducing inference time by more than 7×, thereby validating its effectiveness in handling dynamic scene evolutions under communication latency.

### C. Ablation Study

To evaluate the contribution of each module within the 4D-OVSG framework, an ablation study is conducted on four key components. Specifically: (1) Decision Framework, where the hierarchical planner is replaced by a single VLM for direct task planning; (2) Keyframe Extraction, where the extraction step is removed and the full raw video sequence is processed; (3) Cross-frame Matching, where the MOT-based association is replaced with Hungarian matching as in ST-OVSG; and (4) Spatial Mapping, where the polar-coordinate representation is replaced with VLM-based implicit spatial reasoning. The performance is quantified using Inference Time (IT) and Success Rate (SR). The results are summarized in Table II.

TABLE II
ABLATION RESULTS OF THE PROPOSED 4D-OVSG COMPONENTS.

| Decision | Keyframe | MOT | Polar | Inference Time (s) | Success Rate (SR) |
|---|---|---|---|---|---|
| ✓ | ✓ | ✓ | ✓ | **79.89** | **68.9%** |
| ✗ | ✓ | ✓ | ✓ | 93.14 (+13.25) | 66.7% (−2.2%) |
| ✓ | ✗ | ✓ | ✓ | 102.91 (+23.02) | 65.2% (−3.7%) |
| ✓ | ✓ | ✗ | ✓ | 66.32 (−13.57) | 45.2% (−23.7%) |
| ✓ | ✓ | ✓ | ✗ | 128.54 (+48.65) | 64.4% (−4.5%) |

Table II presents the ablation results of individual components in the proposed 4D-OVSG framework. When the hierarchical *Decision Framework* is removed, inference time increases from 79.89 s to 93.14 s, accompanied by a drop in success rate. This indicates that hierarchical decision-making avoids unnecessary computation and execution of invalid instructions. Removing the *Keyframe Extraction* module leads to a noticeable increase in inference time (from 79.89 s to 102.91 s), confirming that keyframe selection effectively reduces temporal redundancy. Replacing *MOT-based Matching* with Hungarian matching slightly accelerates inference but leads to the largest drop in success rate (-23.7%). This demonstrates that MOT-based approach is more stable than Hungarian matching. Finally, substituting the *spatial Mapping* based on polar-coordinates with implicit spatial reasoning based on VLM leads to the largest increase in inference time (+48.65 s), indicating that geometric encoding offers a more efficient alternative to language-driven spatial inference.

Overall, these results indicate that the effectiveness of 4D-OVSG arises from the complementary contributions of its decision, temporal, tracking, and spatial modeling modules.

## V. CONCLUSION

In this paper, we introduced 4D-OVSG, an embodied multimodal world modeling framework based on 4D scene graphs for long-distance robotic teleoperation. By integrating visual, spatial, and semantic information into a structured spatio-temporal representation, 4D-OVSG bridges perception and decision-making under communication latency, enabling robots to reason over evolving environments despite delayed observations. Extensive experiments demonstrate that this embodied world model achieves a favorable balance between computational efficiency and operational robustness, particularly in complex dynamic scenarios. Beyond the specific teleoperation setting, our results highlight the potential of 4D scene graphs as a general multimodal world modeling paradigm, supporting embodied intelligence that connects perception to planning in real-world robotic systems. Future work may explore integrating online learning mechanisms to continuously adapt the world model and planning strategy from execution feedback.

## REFERENCES


[1] Sidharth Bhanu Kamtam, Qian Lu, Faouzi Bouali, Olivier C. L. Haas, and Stewart Birrell, "Network latency in teleoperation of connected and autonomous vehicles: A review of trends, challenges, and mitigation strategies," *Sensors*, 2024.

[2] Thomas B. Sheridan, *Telerobotics, Automation, and Human Supervisory Control*, MIT Press, 1992.

[3] Günter Niemeyer and Jean-Jacques E. Slotine, "Telemanipulation with time delays," *Int. J. Robot. Res.*, 2004.

[4] Parinaz Farajiparvar, Hao Ying, and Abhilash Pandya, "A brief survey of telerobotic time delay mitigation," *Front. Robot. AI*, 2020.

[5] Haoyu Zhou, Mingyu Ding, Weikun Peng, Masayoshi Tomizuka, Lin Shao, and Chuang Gan, "Generalizable long-horizon manipulations with large language models," *arXiv preprint arXiv:2310.02264*, 2023.

[6] Michael Ahn et al., "Do as I can, not as I say: Grounding language in robotic affordances," *arXiv preprint arXiv:2204.01691*, 2022.

[7] Danny Driess et al., "PaLM-E: An embodied multimodal language model," *arXiv preprint arXiv:2303.03378*, 2023.

[8] Ishika Singh et al., "ProgPrompt: Generating situated robot task plans using large language models," *arXiv preprint arXiv:2209.11302*, 2022.

[9] Jacky Liang et al., "Code as policies: Language model programs for embodied control," *arXiv preprint arXiv:2209.07753*, 2022.

[10] Alec Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. ICML*, 2021.

[11] Shilong Liu et al., "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," in *Proc. ECCV*, 2024.

[12] Alexander Kirillov et al., "Segment anything," in *Proc. ICCV*, 2023.

[13] Youcai Zhang et al., "Recognize anything: A strong image tagging model," in *Proc. CVPR Workshops*, 2024.

[14] Chunlei Wang et al., "OV-VG: A benchmark for open-vocabulary visual grounding," *Neurocomputing*, 2024.

[15] Hans Moravec and Alberto Elfes, "High resolution maps from wide angle sonar," in *Proc. ICRA*, 1985.

[16] Richard A. Newcombe et al., "KinectFusion: Real-time dense surface mapping and tracking," in *Proc. ISMAR*, 2011.

[17] Muzhi Han et al., "Reconstructing interactive 3D scenes by panoptic mapping and CAD model alignments," in *Proc. ICRA*, 2021.

[18] Antoni Rosinol et al., "3D dynamic scene graphs: Actionable spatial perception with places, objects, and humans," *arXiv preprint arXiv:2002.06289*, 2020.

[19] Boyuan Chen et al., "Open-vocabulary queryable scene representations for real-world planning," *arXiv preprint arXiv:2209.09874*, 2022.

[20] Chenguang Huang et al., "Audio-visual language maps for robot navigation," in *Int. Symp. Experimental Robotics*, 2023.

[21] Qiao Gu et al., "ConceptGraphs: Open-vocabulary 3D scene graphs for perception and planning," in *Proc. ICRA*, 2024.

[22] Shan An, Haogang Zhu, Dong Wei, Konstantinos A. Tsintotas, and Antonios Gasteratos, "Fast and incremental loop closure detection with deep features and proximity graphs," *J. Field Robot.*, vol. 39, no. 4, pp. 473–493, 2022.

[23] Yi Wang et al., "Open-vocabulary spatio-temporal scene graph for robot perception and teleoperation planning," *arXiv preprint arXiv:2509.23107*, 2025.

[24] Nicolai Wojke, Alex Bewley, and Dietrich Paulus, "Simple online and realtime tracking with a deep association metric," in *Proc. ICIP*, 2017.

[25] Qwen Team, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.